

Relatório do Trabalho Prático 2

Programação Paralela com CUDA

Êxodo Melo, Thiago Aparecido

20 de Maio de 2026

Resumo

O presente relatório tem o objetivo apresentar o segundo trabalho feito sobre a disciplina de Programação Paralela de paralelização da solução numérica de sistemas lineares utilizando o Método Iterativo de Jacobi utilizando CUDA e seus resultados.

1 Introdução

Sistemas lineares aparecem em diversos locais, desde circuitos elétricos e eletrônicos à sistemas de controle industriais. É de interesse a busca de métodos para solucioná-los.

Um desses métodos é o Método de Jacobi para solução de sistemas lineares, caracterizada pela atualização independente de cada componente do vetor solução a partir dos valores da iteração anterior. A paralelização desse método surge do custo de $O(N^2)$ para cada iteração; tendo independência entre atualizações do vetor solução torna o método particularmente adequado para execução paralela, permitindo explorar arquiteturas de memória compartilhada de forma eficiente.

O método de Jacobi parte da seguinte lógica:

$$\begin{aligned}\sum_j A_{ij}x_j &= b_i \\ A_{ii}x_i + \sum_{j \neq i} A_{ij}x_j &= b_i \\ A_{ii}x_i &= b_i - \sum_{j \neq i} A_{ij}x_j \\ x_i &= \frac{b_i - \sum_{j \neq i} A_{ij}x_j}{A_{ii}}\end{aligned}\tag{1}$$

Tendo uma estimativa inicial para $\mathbf{x}$, pode-se atualizar iterativamente valendo pela equação. Para a convergência do método é precisa que a matriz $\mathbf{A}$ seja uma matriz de diagonal dominante, isto é:

$$|A_{ii}| > \sum_{j \neq i} |A_{ij}|, \forall i$$

A paralelização é um processo difícil e dependente de máquina, uma das maneiras de facilitar a implementação é a abstração de detalhes menos importantes para o programador. Cuda é uma dessas abstrações, qual permite utilizar a arquitetura paralela de GPUs para executar programas computacionalmente intensivos.

Neste trabalho, é apresentada uma implementação do método de Jacobi utilizando CUDA, com o objetivo de explorar o paralelismo de dados e avaliar o impacto da paralelização no desempenho do algoritmo.

É analisado a escolha de número de threads para cada bloco e como organizar as regiões de computação para sua implementação em CUDA, além de detalhes de sincronização e concorrência agora diretamente lidados pelo programador.

Dessa forma, o trabalho busca não apenas demonstrar a aplicação prática de técnicas de paralelização, mas também discutir as limitações e desafios associados à obtenção de ganho de desempenho com a solução paralela.

2 Ambiente Experimental

Os experimentos foram executados no ambiente Google Colab com suporte a GPU NVIDIA, utilizando o compilador `nvcc` (CUDA Toolkit). O código sequencial foi compilado com `g++`. Foi utilizado o parâmetro de compilação `-O3` para otimização pelo compilador.

Foram gerados sistemas lineares consistentes de ordem 1000×1000 , com valores aleatórios para $\mathbf{A}$ e $\mathbf{x}$, garantindo convergência do método de Jacobi. O critério de parada utilizado foi $\epsilon = 0.0001$, com número máximo de 100 iterações.

Para cada configuração de número de threads CUDA, foram realizadas 5 execuções independentes, a fim de reduzir variações de desempenho causadas pelo escalonamento da GPU e processos concorrentes.

Foram geradas sistemas lineares convergentes de valores de $\mathbf{A}$ e $\mathbf{x}$ aleatórios de ordem 1000. Os testes foram executados variando o número de threads, com o objetivo de avaliar o impacto da paralelização no desempenho do método de Jacobi com valor de ϵ fixo sendo $\epsilon = 0.0001$.

O tempo de execução foi medido diretamente na GPU utilizando eventos CUDA (`cudaEvent_t`). O início da medição ocorre imediatamente antes da primeira iteração do algoritmo, enquanto o término é registrado após a finalização do laço iterativo. A diferença entre os eventos é calculada pela função `cudaEventElapsedTime`, que fornece o tempo de execução em milissegundos com alta precisão.

3 Implementação

O algoritmo implementado corresponde ao método iterativo de Jacobi para resolução de sistemas lineares, no qual o vetor solução é atualizado iterativamente a partir dos valores da iteração anterior. A paralelização foi realizada utilizando CUDA, explorando o paralelismo no nível de dados presente nas iterações independentes do método.

A implementação CUDA do método de Jacobi foi desenvolvida explorando o paralelismo de dados presente no algoritmo, onde cada equação do sistema linear é atribuída a uma thread independente. Dessa forma, cada thread é responsável pelo cálculo de uma única componente do vetor solução na iteração atual.

A estrutura geral da implementação é composta por três kernels principais: o kernel de inicialização do vetor solução, o kernel responsável pelo cálculo do novo vetor solução (X_{new}) e o kernel responsável pelo cálculo da norma do erro entre iterações.

No kernel principal do método de Jacobi, cada thread é mapeada diretamente para uma linha da matriz A , realizando o cálculo da seguinte forma:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} A_{ij} x_j^{(k)}}{A_{ii}}$$

Esse mapeamento permite exploração eficiente do paralelismo, pois cada linha da matriz é processada de forma independente. Além, com cada valor de X é iterativamente independente entre si, não é necessário lidar com sincronizações ou concorrência.

O cálculo do erro foi realizado em duas etapas. Primeiramente, cada bloco computa parcialmente a soma das diferenças quadráticas entre X_{new} e X , armazenando o resultado em memória compartilhada. Em seguida, esses valores parciais são transferidos para a CPU, onde é realizada a soma final e o cálculo da norma Euclidiana.

Essa abordagem foi adotada devido à complexidade de realizar uma redução global eficiente diretamente na GPU sem uso de estruturas adicionais de sincronização.

As transferências de dados entre CPU e GPU foram realizadas utilizando `cudaMemcpy`. Inicialmente, a matriz A e o vetor B são transferidos da memória da CPU para a GPU, onde ocorre toda a computação iterativa.

Ao final de cada iteração, o vetor de erro parcial é transferido para a CPU para o cálculo da norma global. Ao término da execução, o vetor solução final X é copiado de volta para a CPU para exibição dos resultados.

Esse padrão reduz a necessidade de comunicação constante entre CPU e GPU, mantendo a maior parte do processamento na GPU.

3.1 Decisões de Projeto

O número de threads por bloco não foi fixado previamente, sendo tratado como um parâmetro experimental. Foram realizados testes variando essa configuração

com o objetivo de avaliar o impacto no desempenho do algoritmo em diferentes níveis de paralelismo.

Os resultados obtidos indicam que o desempenho não cresce de forma linear com o aumento do número de threads por bloco. Esse comportamento é esperado em aplicações CUDA memory-bound, onde o gargalo principal não está no poder de processamento, mas sim na largura de banda de acesso à memória global e na eficiência de uso dos warps.

A grade de execução foi definida com base no número total de elementos do sistema linear e no número de threads por bloco, garantindo cobertura completa do domínio computacional. Para isso, foi utilizada a seguinte formulação:

$$\text{blocks} = \left\lceil \frac{n}{\text{threads}} \right\rceil = \frac{n + \text{threads} - 1}{\text{threads}}$$

Essa abordagem garante que todos os elementos sejam processados mesmo quando o número de elementos não é múltiplo exato do número de threads por bloco, evitando perda de dados no mapeamento entre o vetor e a grade de execução CUDA.

O critério de parada do algoritmo é baseado em dois fatores: o número máximo de iterações e a convergência do método, medida pelo erro calculado. A execução é interrompida quando o erro é menor que um valor limite (ϵ) ou quando o número máximo de iterações é atingido.

As estruturas principais do algoritmo (matriz A , vetores B , X e X_{new}) são armazenadas em memória global da GPU, pois precisam ser acessadas por todas as threads durante a execução.

Foi utilizada memória compartilhada (**shared memory**) no kernel de cálculo do erro para acumular parcialmente a soma da norma Euclidiana dentro de cada bloco, reduzindo a necessidade de acessos repetidos à memória global.

Não foi utilizada memória constante neste trabalho, pois os dados da matriz são dinâmicos e variam conforme o sistema linear de entrada. Apesar de ser possível utilização para a matriz A e B , não foi utilizado pela busca de simplicidade.

4 Resultados

4.1 Desempenho

Os experimentos foram realizados variando o número de threads utilizadas na execução paralela do método de Jacobi. Para cada configuração, foram realizadas cinco execuções, sendo considerado o tempo médio.

O speedup foi calculado em relação à execução sequencial (1 thread), conforme a Equação:

$$S(p) = \frac{T(1)}{T(p)}$$

A eficiência foi calculada como:

$$E(p) = \frac{S(p)}{p}$$

Abaixa a Tabela 1 de dados sobre tempo, speedup e eficiência contra o programa paralelo com 1 thread.

Num. Threads por bloco	Tempo (ms)	Speedup (vs. 1 thread)	Eficiência
1	11.925	-	-
2	9.996	1.19	0.59
4	6.359	1.88	0.47
8	6.070	1.96	0.25
12	6.476	1.84	0.15
16	8.326	1.43	0.09

Table 1: Desempenho do algoritmo CUDA variando número de threads

Abaixo, os gráficos para cada medida contra o Numero de Threads.

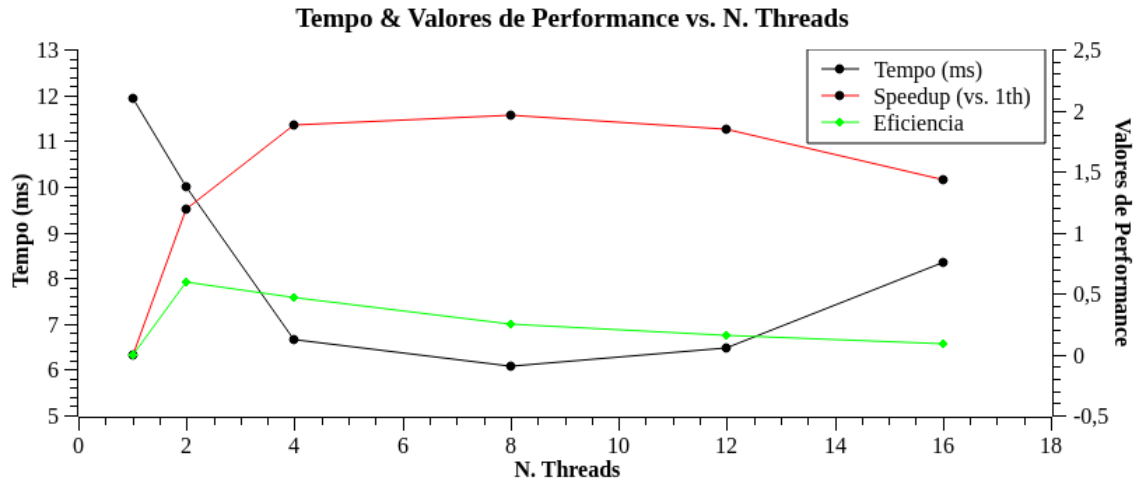


Figure 1: Tempo & Valores de Performance vs. Num. Threads

Abaixa a Tabela 2 de dados sobre speedup e eficiência contra o programa sequencial e contra o OpenMp

Versão	Configuração	Tempo (ms)	Speedup (vs. Seq.)	Speedup (vs. OpenMp.)
Seq.	-	134.634	-	-
OpenMP.	16 threads	6.64	20.27	-
GPU	1 threads/bloco	11.925	11.27	0.55
GPU	2 threads/bloco	9.996	13.47	0.66
GPU	4 threads/bloco	6.359	21.17	1.04
GPU	8 threads/bloco	6.070	22.18	1.09
GPU	12 threads/bloco	6.476	20.78	1.02
GPU	16 threads/bloco	8.326	16.17	0.79

Table 2: Comparação entre execução sequencial, openMP e CUDA

Abaixo o gráfico de Speedup de ambas implementações contra a implementação em CUDA variando o numero de threads.

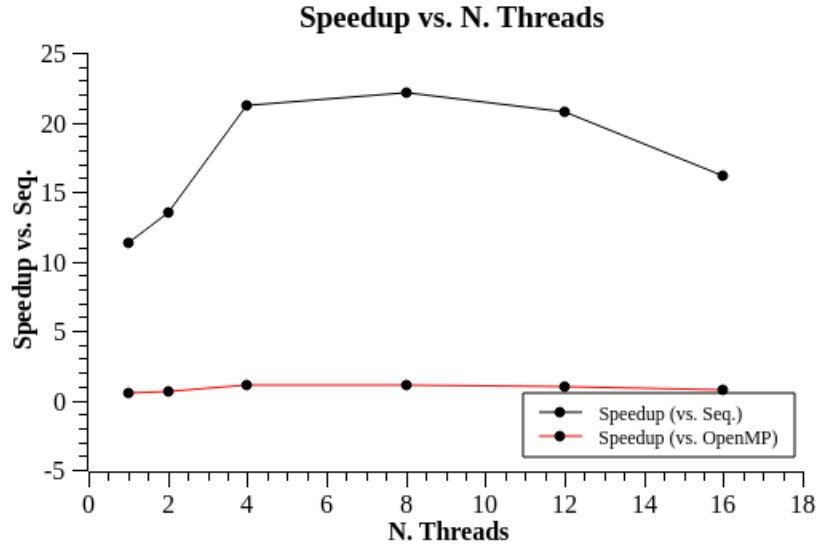


Figure 2: Speedup vs. Num. Threads

Observa-se uma redução significativa no tempo de execução ao utilizar a paralelização via CUDA em relação à versão sequencial, com speedup superior a 20x na melhor configuração (8 threads por bloco).

Em comparação com a implementação em OpenMP, os resultados mostram desempenhos próximos entre as abordagens paralelas, com leve vantagem para a GPU em configurações intermediárias (4 a 8 threads por bloco), enquanto configurações extremas apresentam degradação de desempenho.

Entretanto, o aumento do número de threads por bloco não resulta em ganho linear de desempenho. A partir de 8 threads, observa-se estabilização ou redução de desempenho, o que pode estar associado ao overhead de organização dos

warps, padrões de acesso à memória global e limitação de ocupação eficiente da GPU para o tamanho do problema analisado.

Além disso, o comportamento do algoritmo indica predominância de limitações relacionadas à memória (memory-bound), o que restringe a escalabilidade do desempenho mesmo com o aumento do grau de paralelismo.

4.2 Convergência

A convergência do método de Jacobi depende diretamente das propriedades da matriz de coeficientes. Em particular, uma condição suficiente para a convergência é que a matriz seja diagonalmente dominante. No entanto, nem todos os sistemas utilizados nos experimentos satisfizeram essa propriedade.

Durante a execução dos testes, foram observados casos em que o método não convergiu. Nessas situações, o erro entre iterações não apresentou comportamento decrescente e, em alguns casos, cresceu progressivamente ao longo das iterações. Esse comportamento pode levar à divergência numérica, fazendo com que os valores do vetor solução assumam magnitudes muito elevadas, resultando em valores infinitos (`inf` ou `-inf`).

O critério de parada adotado foi baseado na norma Euclidiana da diferença entre iterações consecutivas:

$$\|X^{(k+1)} - X^{(k)}\|_2$$

Nos casos em que o método converge, esse erro tende a zero à medida que o número de iterações aumenta. Por outro lado, nos casos divergentes, o erro tende a crescer ou oscilar, impedindo que o critério de parada baseado em ϵ seja satisfeito.

Além disso, é importante destacar que a paralelização do algoritmo não influencia a convergência do método, pois não altera sua formulação matemática. A divergência observada está ligada exclusivamente às características do sistema linear e não à utilização de múltiplas threads.

Dessa forma, conclui-se que a garantia de convergência do método de Jacobi depende fortemente da construção da matriz de coeficientes, sendo fundamental propriedades como dominância diagonal para aplicações práticas.

É possível realizar alterações na forma qual a tolerância é disposta a depender do número de threads. Entretanto, não realizado tal implementação nesse trabalho, mantendo a tolerância igual para todas threads.

Abaixo temos a uma Tabela 3 iterações vs erro para a execução do algoritmo de um sistemas de ordem 1000, número de iterações 300 e $\epsilon = 0.0001$ sendo o sistema gerado convergente.

Iteração	Erro (Sistema 01)
0	322,77
50	304,22
100	298,2
200	286,49
299	275,36

Table 3: Erro dos sistemas para cada iteração

4.3 Discussão

Os resultados obtidos demonstram que a utilização de CUDA foi eficaz para reduzir significativamente o tempo de execução do método de Jacobi em comparação com a versão sequencial. O melhor desempenho observado na GPU foi de aproximadamente 6.07 ms, representando um speedup superior a 20x em relação à implementação sequencial, que apresentou tempo médio de 134.63 ms.

O método de Jacobi apresenta características naturalmente adequadas para paralelização, uma vez que o cálculo de cada componente do vetor solução em uma iteração depende apenas dos valores da iteração anterior. Dessa forma, foi possível mapear cada equação do sistema para execução concorrente na GPU.

Entretanto, observou-se que o aumento do número de threads por bloco não resulta em ganho linear de desempenho. O desempenho melhora até uma configuração intermediária (8 threads por bloco), após a qual ocorre estabilização ou leve degradação.

Esse comportamento pode ser explicado pelas características da arquitetura CUDA. O algoritmo apresenta intenso acesso à memória global, caracterizando-se como memory-bound. Assim, o gargalo principal não está no poder de processamento, mas na largura de banda e no padrão de acesso à memória.

Além disso, o desempenho também é influenciado pelo escalonamento em warps e pela ocupação da GPU, de forma que configurações intermediárias tendem a melhor aproveitar os recursos disponíveis.

Mesmo com essas limitações, os resultados demonstram que a utilização de CUDA proporciona ganhos expressivos de desempenho para o método de Jacobi, especialmente em comparação com a versão sequencial. Em relação à implementação em OpenMP, observa-se desempenho competitivo, com a GPU apresentando vantagem em algumas configurações intermediárias, mas sem superioridade absoluta em todos os casos.

5 Conclusão

Neste trabalho foi desenvolvida uma implementação paralela do método iterativo de Jacobi utilizando CUDA, explorando o paralelismo massivo oferecido pela arquitetura das GPUs NVIDIA.

Os resultados experimentais demonstraram uma redução significativa no tempo de execução em comparação com a versão sequencial, alcançando speedup

superior a 20x na melhor configuração avaliada. Os testes evidenciaram que o método de Jacobi é adequado para paralelização em GPU devido à independência entre os cálculos das componentes do vetor solução em cada iteração.

Também foi possível observar que o desempenho da aplicação depende diretamente da configuração de execução adotada, especialmente do número de threads por bloco. Foi identificado um ponto ótimo de desempenho em torno de 8 threads por bloco, enquanto configurações maiores não resultaram em ganhos adicionais relevantes.

Além disso, verificou-se que o comportamento do algoritmo é fortemente influenciado por limitações de memória (memory-bound), o que restringe a escalabilidade do desempenho mesmo com aumento do paralelismo.

Em comparação com a implementação em OpenMP, os resultados indicam que ambas as abordagens paralelas apresentam desempenhos competitivos, dependendo da configuração utilizada.

Dessa forma, conclui-se que a utilização de CUDA é uma abordagem eficiente para aceleração do método de Jacobi, especialmente em problemas de grande porte, demonstrando o potencial das GPUs para computação científica e processamento numérico paralelo.

6 Referencias

- [1] Vipin Kumar, Ananth Grama, Anshul Gupta, and George Karypis. *Introduction to parallel computing*, volume 110. Benjamin/Cummings Redwood City, CA, 1994.
- [2] Philip Wilkinson. *Parallel programming: Techniques and applications using networked workstations and parallel computers*, 2/E. Pearson Education India, 2006.